



Patrón Singleton en Python

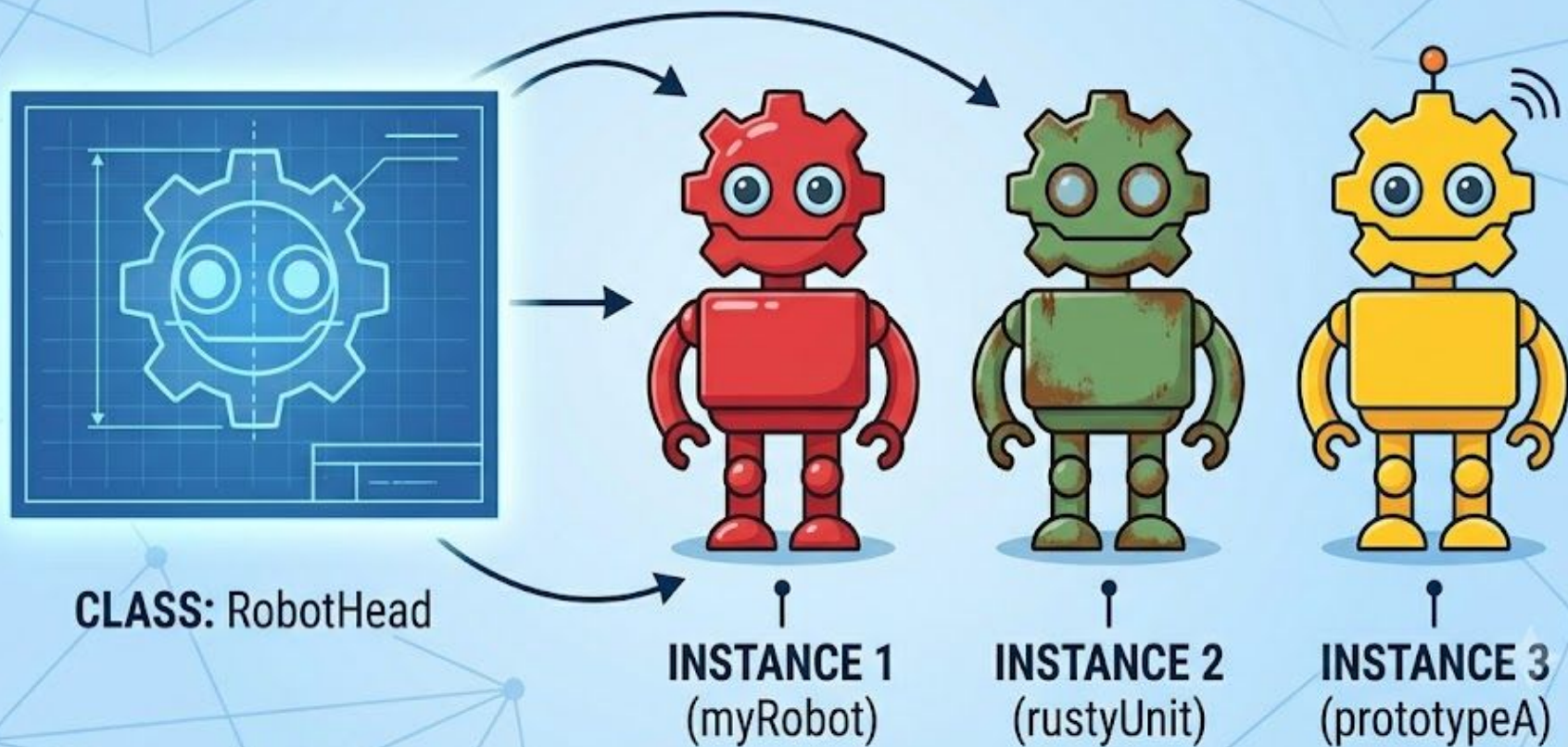
Gomez Simon - Leonardo Saucedo - Martina Sturma - Jackeline Santolere

Qué es una instancia?

Una instancia es un objeto único creado a partir de una clase. Si la clase es «Robot», la instancia es el objeto real construido en memoria. Cada instancia tiene sus propios datos y puede ejecutar las acciones definidas en la clase. La creación de objetos a partir de clases es un concepto clave.

Un objeto se refiere a cualquier entidad que tenga propiedades y métodos, mientras que una instancia es específicamente un objeto creado a partir de una clase. Por ejemplo, si tenemos una clase llamada «Robot», un objeto llamado «miRobot» creado a partir de esta clase, sería una instancia de «Robot».

OBJECT INSTANCE



instancia.py X

instancia > instancia.py > ...

```
1 from robot import Robot
2
3 robot1 = Robot("head1","body1","legs1","feet")
4 robot2 = Robot("head2","body2","legs2","feet")
5 robot3 = Robot("head3","body3","legs3","feet")
6 print(robot1)
7 print(robot2)
8 print(robot3)
9
10
```

robot.py X

instancia > robot.py > Robot > feet

```
1 class Robot():
2
3     def __init__(self,head,body,legs,feet):
4         self.__head = head
5         self.__body = body
6         self.__legs = legs
7         self.__feet = feet
8
9     @property
10    def head(self):
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

+ v ... | {} X

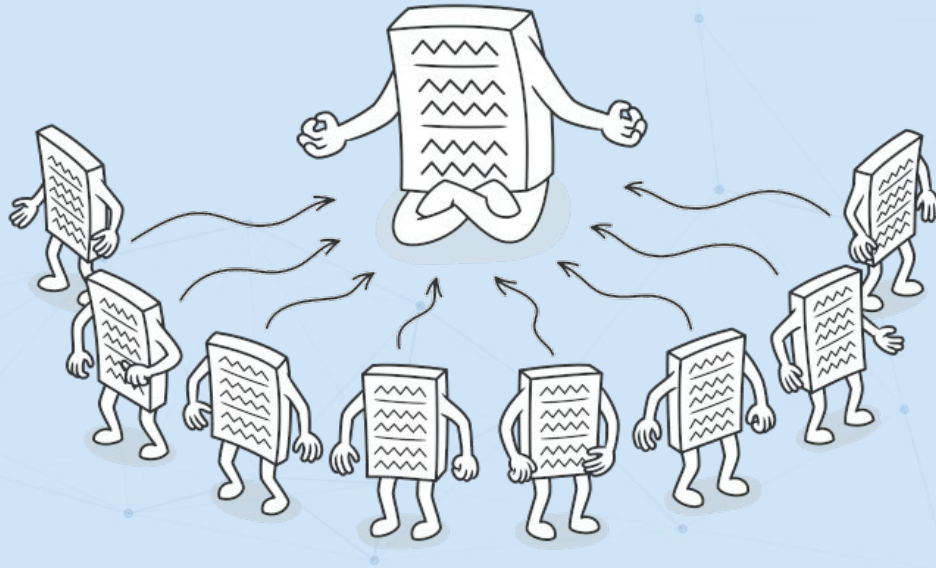
```
PS I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2> ^C
PS I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2>
PS I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2> i;; cd 'i:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2'; & 'C:\Users\Rockstone\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\Rockstone\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '63427' '--' 'I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2\instancia\instancia.py'
<robot.Robot object at 0x000001FC7F393E00>
<robot.Robot object at 0x000001FC7F3BA210>
<robot.Robot object at 0x000001FC7F3BA350>
PS I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2>
```

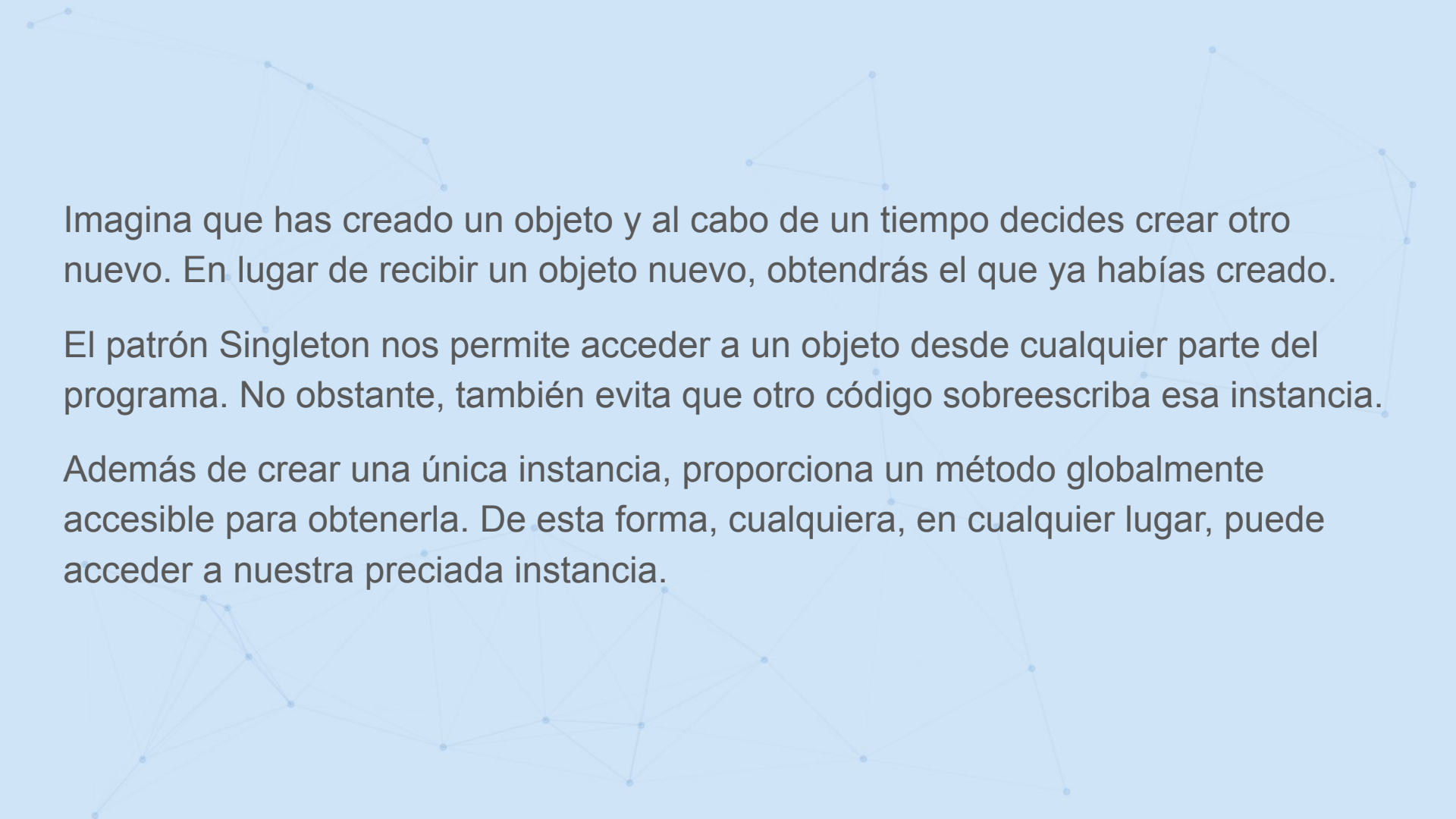
powersh...

Python Deb...

Patrón Singleton:

Singleton es un patrón de diseño creacional que nos permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.

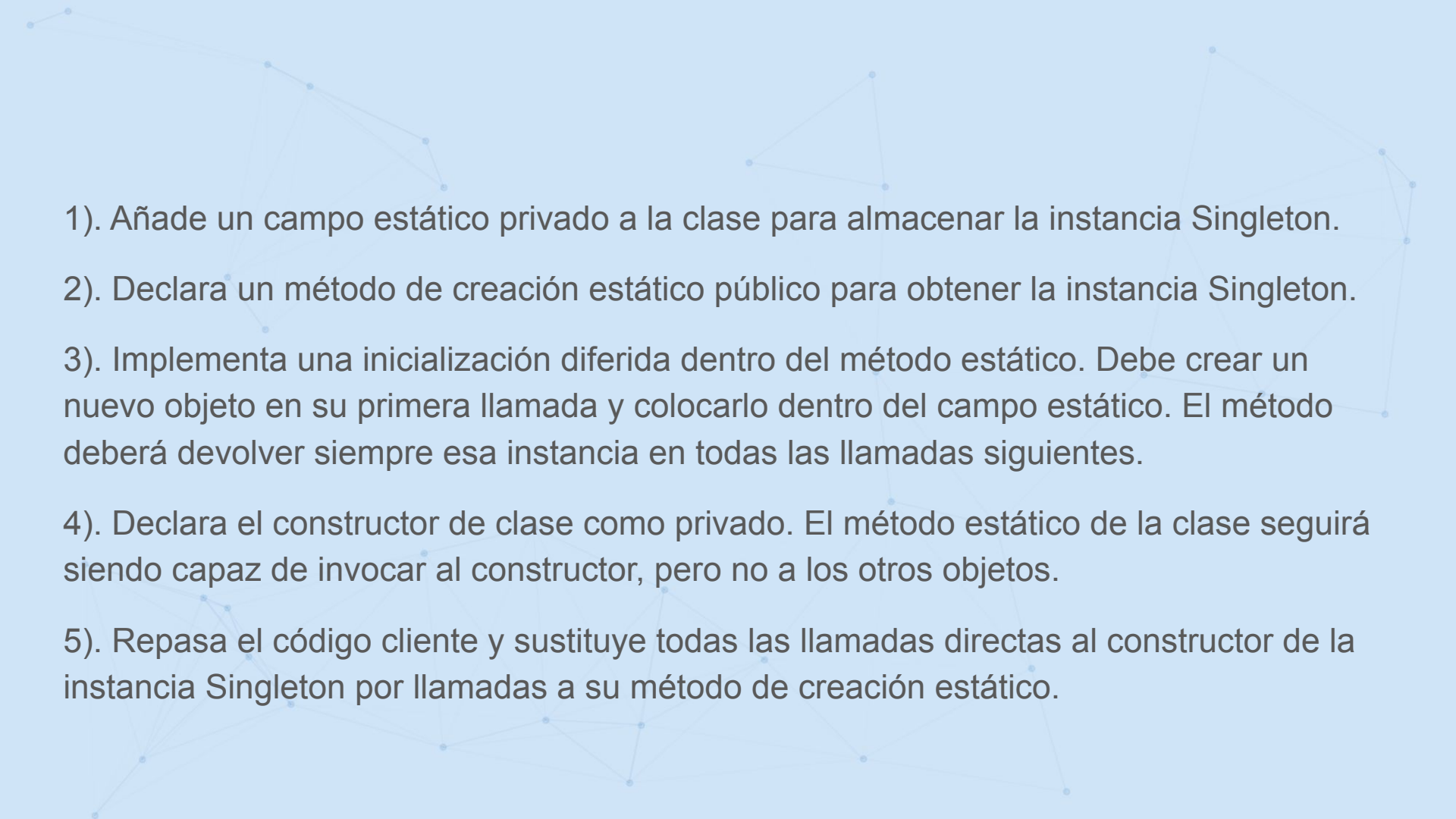




Imagina que has creado un objeto y al cabo de un tiempo decides crear otro nuevo. En lugar de recibir un objeto nuevo, obtendrás el que ya habías creado.

El patrón Singleton nos permite acceder a un objeto desde cualquier parte del programa. No obstante, también evita que otro código sobrescriba esa instancia.

Además de crear una única instancia, proporciona un método globalmente accesible para obtenerla. De esta forma, cualquiera, en cualquier lugar, puede acceder a nuestra preciada instancia.

- 
- 1). Añade un campo estático privado a la clase para almacenar la instancia Singleton.
 - 2). Declara un método de creación estático público para obtener la instancia Singleton.
 - 3). Implementa una inicialización diferida dentro del método estático. Debe crear un nuevo objeto en su primera llamada y colocarlo dentro del campo estático. El método deberá devolver siempre esa instancia en todas las llamadas siguientes.
 - 4). Declara el constructor de clase como privado. El método estático de la clase seguirá siendo capaz de invocar al constructor, pero no a los otros objetos.
 - 5). Repasa el código cliente y sustituye todas las llamadas directas al constructor de la instancia Singleton por llamadas a su método de creación estático.

main.py

singleton > main.py > ...

```
1 from robot import Robot
2 if __name__ == "__main__":
3     robot1 = Robot()
4     robot2 = Robot()
5
6     print(robot1) # Output: <__main__.Robot object at 0x000001AC40A63E00>
7     print(robot2)
8     print(robot1 is robot2) # Output: True
9     print(robot1.name) # Output: Robot
10    print(robot2.name) # Output: Robot
11    robot1.name = "Robo1"
12    print(robot1.name) # Output: Robo1
13    print(robot2.name) # Output: Robot
14
```

robot.py

singleton > robot.py > Robot

```
1 class Robot():
2     instance = None
3
4     def __new__(cls):
5         if cls.instance is None:
6             cls.instance = super().__new__(cls)
7         return cls.instance
8
9     def __init__(self, name="Robot"):
10         self.__name = name
11
12     @property
13     def name(self):
14         return self.__name
15
16     @name.setter
17     def name(self, name):
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

```
PS I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2> & 'C:\Users\Rockstone\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\Rockstone\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52891' '--' 'I:\tinynotes\0_TUP_UTN\Programacion II\5_python\TP2\singleton\main.py'
```

<robot.Robot object at 0x000001AC40A63E00>

<robot.Robot object at 0x000001AC40A63E00>

True

Robot

Robot

Robo1

Robo1

powersh...

Python Deb...

Otra forma muy usada en Python, aprovechando que los módulos ya son singletons por naturaleza (un módulo solo se importa una vez por proceso), es directamente definir el estado a nivel de módulo en vez de usar una clase

 robot_3.py X

moduled >  robot_3.py > ...

```
2  robot = None
```

```
3
```

```
4  def setRobot(data):
```

```
5      robot = data
```

```
6
```

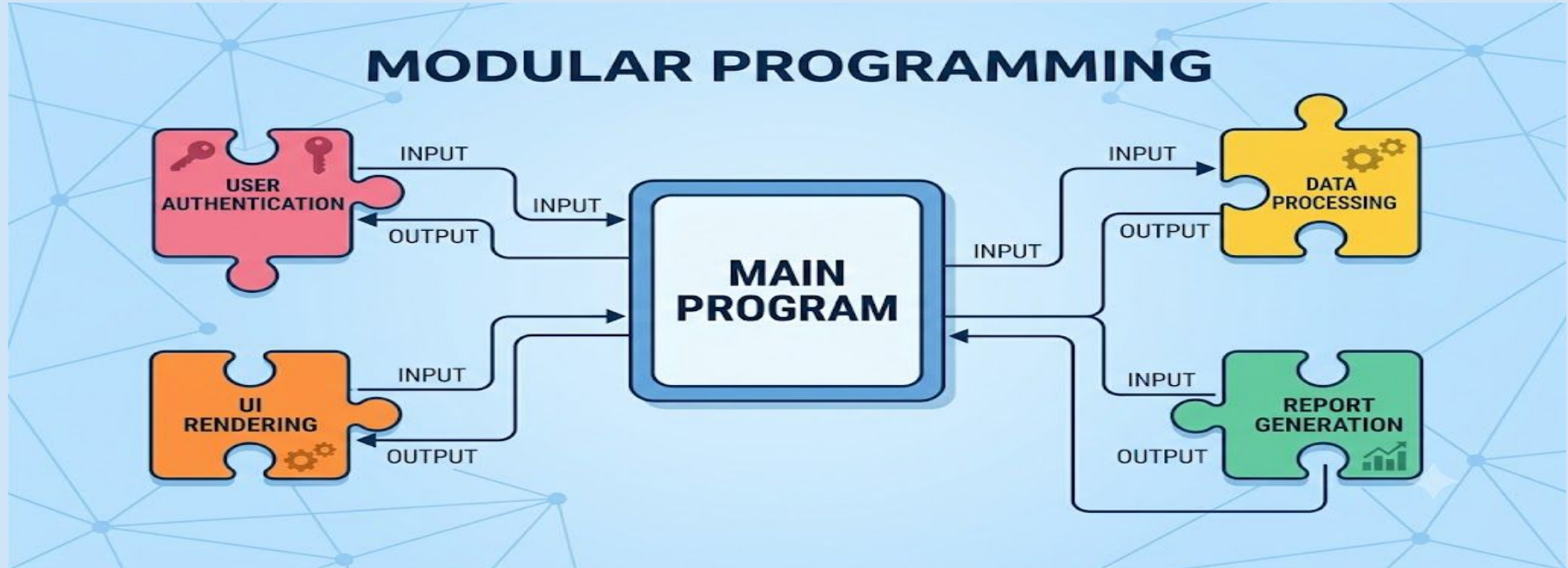
```
7  def getRobot():
```

```
8      return robot
```

```
9
```

Programación Modular:

La programación modular es un paradigma que divide un problema complejo en bloques independientes manejables y más pequeños llamados módulos.



La programación modular es el principio general de dividir el sistema en piezas independientes, y el Singleton es una herramienta puntual que usás dentro de ese diseño cuando necesitás que una de esas piezas tenga garantizada una única instancia compartida por todos los módulos que la consumen.

python

```
class ConfigManager:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.config = {}
        return cls._instance
```